

DA2304: Introduction to Computer Systems

Tutorial 1: Building Logic Gates

Data Science and AI Department (DSAI)
Indian Institute of Technology Madras

Instructor: Dr. Patanjali SLPSK (patanjali@dsai.iitm.ac.in)

Release Date: February 2, 2026

Deadline: February 2, 2026, 11:59 PM

Introduction

The tutorial will introduce you to the NAND2Tetris toolchain. For the first module, we will be using the Hack Hardware Programming language. In this tutorial, you will learn how to design basic gates using the hack programming language.

Use the ide at NAND2Tetris Web IDE to implement, test, and submit your designs.

- Use Chromium based browsers to enable local file storage.

Note: All gates must be built using only the NAND gate as the primitive building block, unless otherwise specified in later exercises.

NAND gate is a universal gate. The hack interface supports the NAND operation by default and any other 2-input boolean gates will be built on top of the NAND gates.

Prerequisites

1. Access to the NAND2Tetris Web IDE
2. Basic understanding of Boolean algebra and truth tables (Lectures 2,3, and 4).

1 Exercise 1: Basic Gates

The syntax for the HDL used for this assignment is uploaded along with the tutorial problem statement.

1.1 AND Gate

Specification:

- Inputs: a, b (1-bit each)
- Output: out (1-bit)
- Function: $out = a \cdot b$ (Boolean AND)
- Truth Table:

a	b	out
0	0	0
0	1	0
1	0	0
1	1	1

Solution

```

1 CHIP And {
2     IN a, b;
3     OUT out;
4     PARTS:
5     Nand(a=a, b=b, out=nandOut);
6     Not(in=nandOut, out=out);
7 }
```

Explanation: The AND gate can be implemented as $NOT(NAND(a,b))$.

1.2 NOT Gate

Specification:

- Input: in (1-bit)
- Output: out (1-bit)
- Function: $out = \overline{in}$ (Boolean NOT)

Solution

```

1 CHIP Not {
2     IN in;
3     OUT out;
4     PARTS:
5     Nand(a=in, b=in, out=out);
6 }
```

Explanation: $NAND(a,a) = NOT(a)$ since $\overline{a \cdot a} = \overline{a}$.

2 Exercise 2: Composite Gates

Design the following gates using any combination of basic gates (AND, OR, NOT, NAND) already built.

2.1 XOR Gate (Exclusive OR)

Specification:

- Inputs: a, b (1-bit each)
- Output: out (1-bit)
- Function: $out = a \oplus b = a\bar{b} + \bar{a}b$
- Truth Table:

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Solution

```

1 CHIP Xor {
2     IN a, b;
3     OUT out;
4     PARTS:
5     Nand(a=a, b=b, out=nand1);
6     Nand(a=a, b=nand1, out=nand2);
7     Nand(a=nand1, b=b, out=nand3);
8     Nand(a=nand2, b=nand3, out=out);
9 }

```

Alternative Implementation (if you have a OR implementation defined):

```

1 CHIP Xor {
2     IN a, b;
3     OUT out;
4     PARTS:
5     Not(in=a, out=notA);
6     Not(in=b, out=notB);
7     And(a=a, b=notB, out=and1);
8     And(a=notA, b=b, out=and2);
9     Or(a=and1, b=and2, out=out);
10 }

```

3 Testing Guidelines

1. Use the **test script** feature in the Web IDE to verify each gate's truth table.
2. Test all possible input combinations for each gate.
3. Ensure your designs are **efficient** (use minimal gates where possible).
4. **Do not** use built-in chips (e.g., built-in And, Or) except NAND and NOT in Exercise 1.
5. For multi-bit gates, test with various input patterns (all zeros, all ones, alternating patterns).

4 Homework

1. Implement OR, NOR and XNOR gates using the following constraints
 - Implement OR using AND and NOT gates
 - Implement NOR using NAND and NOT gates
 - Implement XNOR using OR and NOT gates
2. Minimize the boolean expression to use fewer gates (assume all gates have equal delay)

$$y = AB'C + A'BC + ABC' + A'B'C + AB'C' \quad (1)$$

3. Implement the minimal expression from previous question using the hack programming language.

5 Submission Instructions

1. The Answers for question 2 should be submitted as a pdf titled Name_rollno.pdf.
2. For questions 1, 3 please follow instructions below:
 - Save each HDL file as GateName.hdl (e.g., Xnor.hdl, And4.hdl).
 - Test each file individually in the Web IDE.
3. Compress all Name_rollno.pdf, GateName.hdl, GateName.cmp, GateName.tst files into a **ZIP archive** named:

DA2304_Tutorial1_RollNumber.zip

4. Files will be autograded, keep the names as instructed.
5. Upload the archive to the course portal by February 2, 2026, 11:59 PM.
6. Late submissions will be penalized as per course policy.

Academic Integrity

- You may discuss concepts with classmates but must write your own code.
- Do not share HDL files or copy code from others, nor use AI Assistive tools.
- Cite any external resources used.
- Violations will be dealt with as per IIT Madras academic integrity policies.

Good luck! For queries, post on the course forum or contact the TAs.